



त्रिभुवन विश्वविद्यालय
Tribhuvan University

Patan Multiple Campus

Bachelor in Information Technology (BIT)

Practical file

Subject : Introduction to Information Technology
Course No : BIT101
Batch : 2083
Semester : 1st

Submitted To:
Deo N. Yadav
(Associate Professor)

Submitted by:
Name: Saroj Regmi
Roll No: 06

1. What are the boot files of DOS operating system. Explain the function of each.

Boot files are special system files that are required to start or “boot” an operating system when a computer is turned on. They contain the instructions and components needed to load the operating system into memory and make the computer ready for use. The main boot files of MS-DOS are:

1. IO.SYS

- 1.1 It is the DOS input/output system file.
- 1.2 It contains basic device drivers and routines needed to communicate with hardware.
- 1.3 It initializes the system and provides basic input/output services.

2. MSDOS.SYS

- 2.1 It is the DOS kernel.
- 2.2 It manages important operating-system functions such as file management, memory management, and program execution.
- 2.3 It works with IO.SYS to provide the basic DOS environment.

3. COMMAND.COM

- 3.1 It is the DOS command interpreter.
- 3.2 It displays the command prompt and interprets commands such as DIR, COPY, DEL, and CD.
- 3.3 It also executes batch files such as AUTOEXEC.BAT.

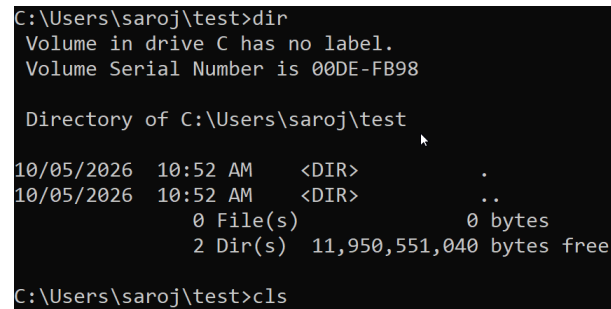
2. Write the function of following commands and their syntax in the lab sheet. Run and test each command in Lab.

2.1. cls

cls command is used to clear the screen.

syntax :

```
cls
```



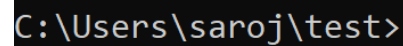
```
C:\Users\saroj\test>dir
Volume in drive C has no label.
Volume Serial Number is 00DE-FB98

Directory of C:\Users\saroj\test

10/05/2026  10:52 AM    <DIR>          .
10/05/2026  10:52 AM    <DIR>          ..
               0 File(s)                0 bytes
               2 Dir(s)  11,950,551,040 bytes free

C:\Users\saroj\test>cls
```

Figure 1: Before using the `cls` command



```
C:\Users\saroj\test>
```

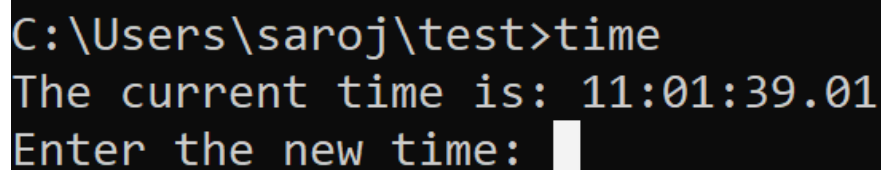
Figure 2: After using the cls command

2.2. time

time command is used to display or set the system time. If used without parameters, it displays the current system time and prompts you to enter a new time.

syntax :

```
time [/t | [<HH>[:<MM>[:<SS>]] [am|pm]]]
```



```
C:\Users\saroj\test>time
The current time is: 11:01:39.01
Enter the new time: 
```

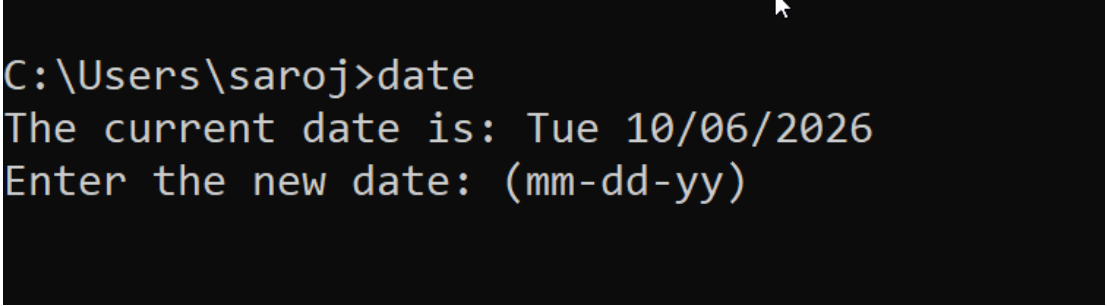
Figure 3: An example of time command.

2.3. date

date command is used to display or set the system date. If used without parameters, it displays the current system date and prompts you to enter a new date. It is very similar to the time command above.

syntax :

```
date [/t | <month-day-year>]
```

A screenshot of a Windows command prompt window. The prompt is 'C:\Users\saroj>'. The user has entered 'date'. The output is 'The current date is: Tue 10/06/2026' followed by 'Enter the new date: (mm-dd-yy)'.

```
C:\Users\saroj>date
The current date is: Tue 10/06/2026
Enter the new date: (mm-dd-yy)
```

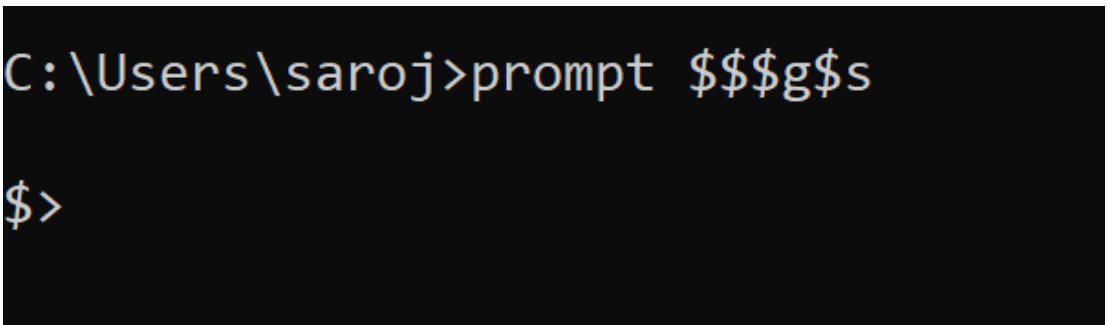
Figure 4: An example of date command.

2.4. prompt

prompt command is used to set the current prompt syntax, which will get expanded when the user is using the cmd. The line of text that is present before the user types the command in cmd is called prompt. It is generally in the format of %p\$g which expands to current directory and a greater than symbol ">". as shown in the figure below. This prompt can be changed into anything you like be it current directory and = symbol to only your name. we can explore the available prompt options using the command prompt /?

syntax :

```
prompt [<text>]
```

A screenshot of a Windows command prompt window. The prompt is 'C:\Users\saroj>'. The user has entered 'prompt \$\$\$g\$s'. The output is '\$>'.

```
C:\Users\saroj>prompt $$$g$s
$>
```

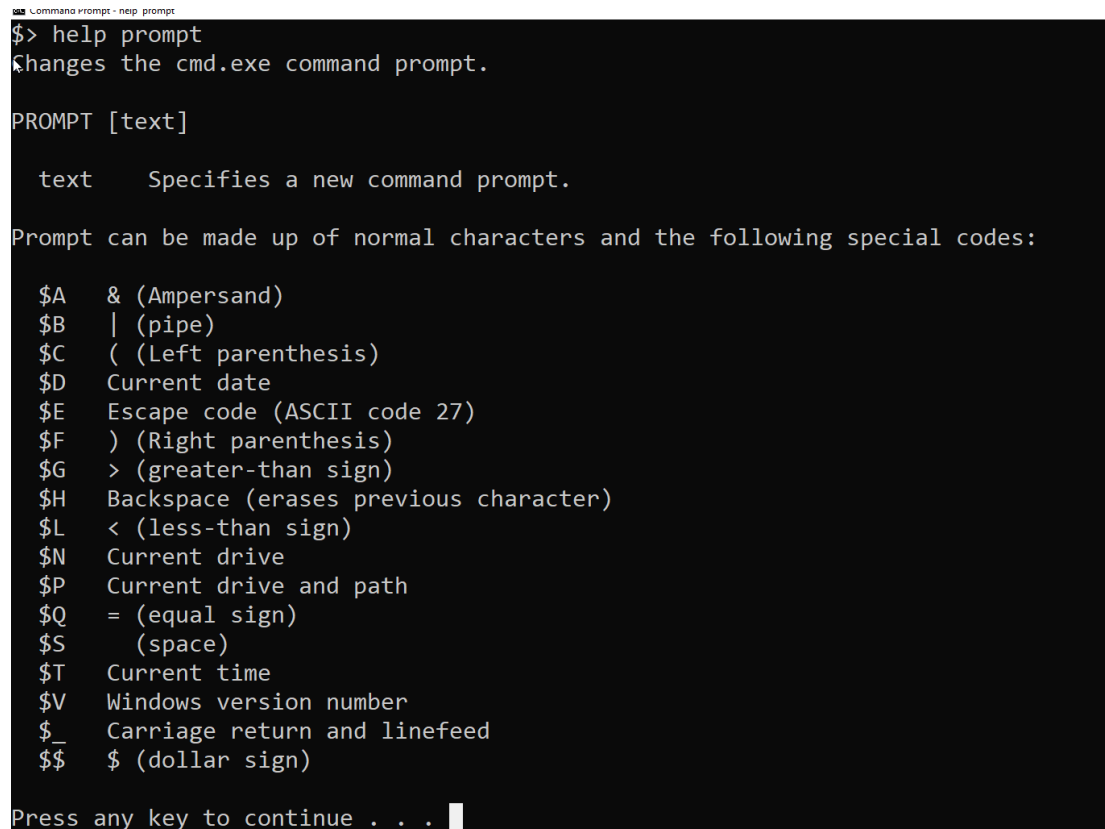
Figure 5: An example of setting prompt very minimal popular unix prompt \$> .

2.5. help

help command displays a list of the available commands or detailed help information on a specified command. If used without parameters, help lists and briefly describes every system command.

syntax :

```
help [<command>]
```



```
Command Prompt - help prompt
$> help prompt
Changes the cmd.exe command prompt.

PROMPT [text]

text    Specifies a new command prompt.

Prompt can be made up of normal characters and the following special codes:

$A  & (Ampersand)
$B  | (pipe)
$C  ( (Left parenthesis)
$D  Current date
$E  Escape code (ASCII code 27)
$F  ) (Right parenthesis)
$G  > (greater-than sign)
$H  Backspace (erases previous character)
$L  < (less-than sign)
$N  Current drive
$P  Current drive and path
$Q  = (equal sign)
$S  (space)
$T  Current time
$V  Windows version number
$_  Carriage return and linefeed
$$  $ (dollar sign)

Press any key to continue . . .
```

Figure 6: An example of using help command to see help info about prompt command.

3. What is wildcard? What are two wildcard characters. Explain with example.

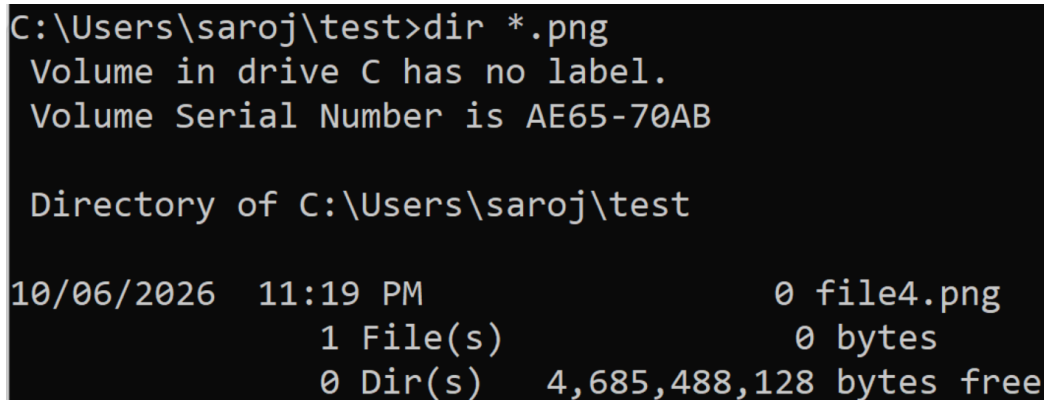
In software or generally, a wildcard character is a special single character that serves as a placeholder for some other character or characters. There are two wildcard characters in DOS.

They are:

1. asterisk (*)
2. question mark (?)

1. asterisk (*)

asterisk is a wildcard character that serves as a placeholder for any number of characters and matches every single character.



```
C:\Users\saroj\test>dir *.png
Volume in drive C has no label.
Volume Serial Number is AE65-70AB

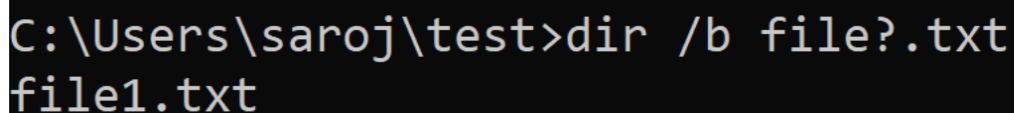
Directory of C:\Users\saroj\test

10/06/2026  11:19 PM                0 file4.png
               1 File(s)                0 bytes
               0 Dir(s)  4,685,488,128 bytes free
```

Figure 7: An example of using **asterisk** wildcard.

2. question mark (*)

question mark is a wildcard character that serves as a placeholder for only one character and matches only one character



```
C:\Users\saroj\test>dir /b file?.txt
file1.txt
```

Figure 8: An example of using **question mark** wildcard.

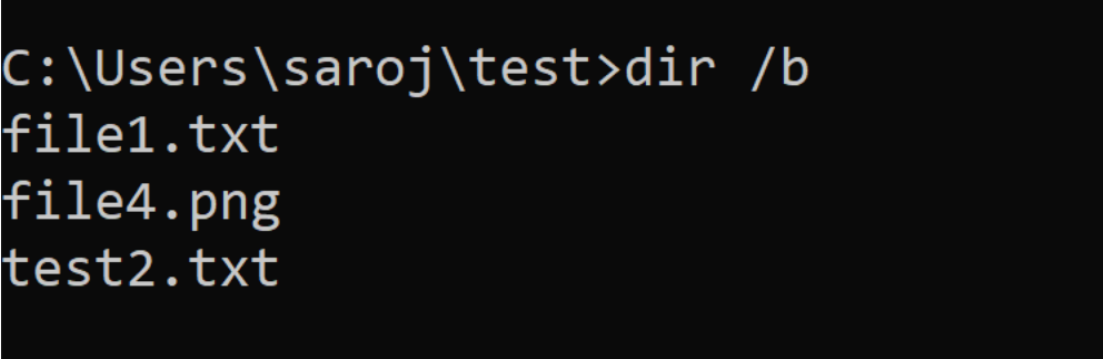
4. Directory Manipulation:

4.1 **dir**

dir command is used to list the files and directory of the current location or path.

syntax :

```
dir
```



```
C:\Users\saroj\test>dir /b
file1.txt
file4.png
test2.txt
```

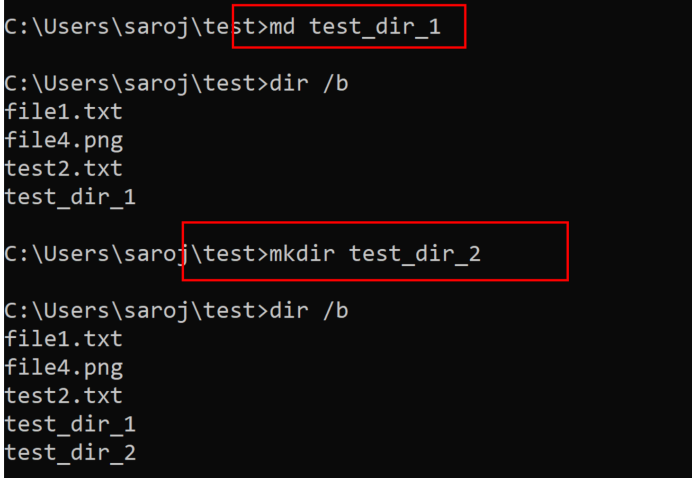
Figure 9: An example of using **dir** command.

4.2 **md** or **mkdir**

md or **mkdir** command is used to make a directory.

syntax :

```
mkdir [<drive>:]<path>
md [<drive>:]<path>
```



```
C:\Users\saroj\test>md test_dir_1
C:\Users\saroj\test>dir /b
file1.txt
file4.png
test2.txt
test_dir_1
C:\Users\saroj\test>mkdir test_dir_2
C:\Users\saroj\test>dir /b
file1.txt
file4.png
test2.txt
test_dir_1
test_dir_2
```

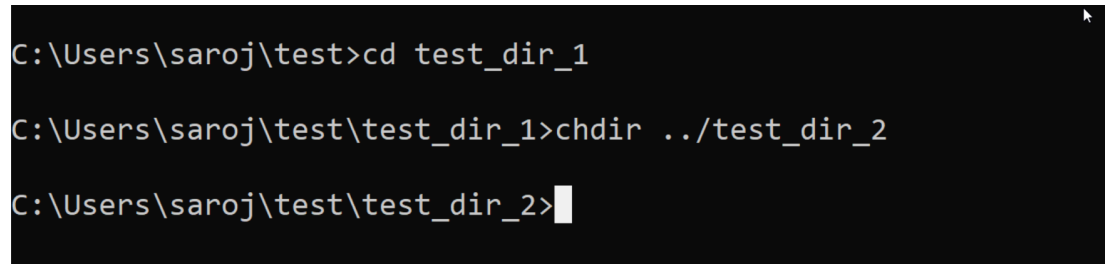
Figure 10: An example of using **md** and **mkdir** command.

4.3 **cd** or **chdir**

cd or **chdir** command is used change the current directory.

syntax :

```
cd [/d] [<drive>:][<path>]
cd [..]
chdir [/d] [<drive>:][<path>]
chdir [..]
```



```
C:\Users\saroj\test>cd test_dir_1
C:\Users\saroj\test\test_dir_1>chdir ../test_dir_2
C:\Users\saroj\test\test_dir_2>
```

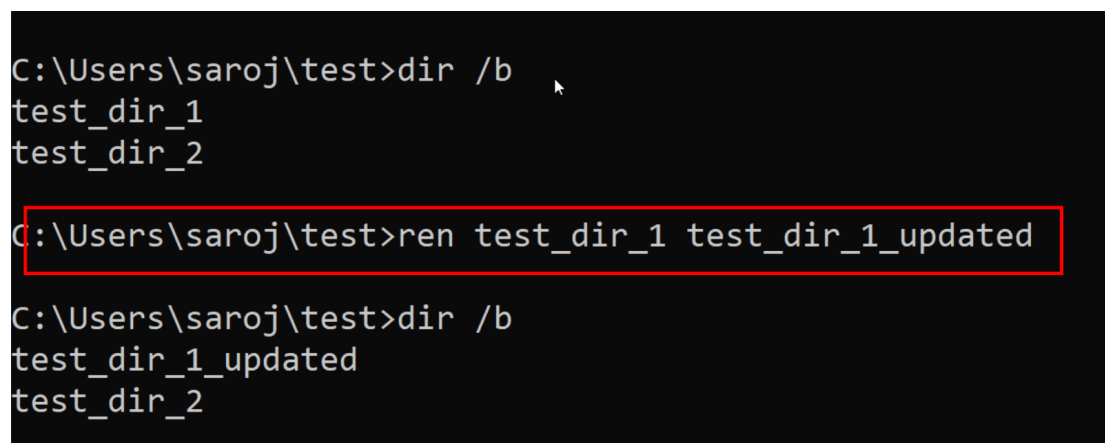
Figure 11: An example of using **chdir** and **cd** command.

4.4 **ren**

ren command is used to rename file and directory in cmd.

syntax :

```
ren [<drive>:][<path>]<filename1> <filename2>
```



```
C:\Users\saroj\test>dir /b
test_dir_1
test_dir_2

C:\Users\saroj\test>ren test_dir_1 test_dir_1_updated

C:\Users\saroj\test>dir /b
test_dir_1_updated
test_dir_2
```

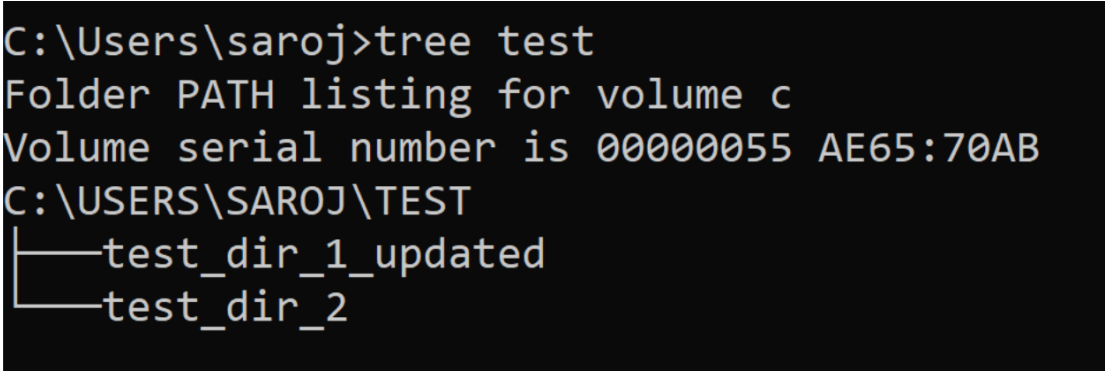
Figure 12: An example of using **ren** command.

4.5 tree

tree command is used to display the directory structure of a path or of the disk in a drive graphically.

syntax :

```
tree [<drive>:][<path>] [/f] [/a]
```



```
C:\Users\saroj>tree test
Folder PATH listing for volume c
Volume serial number is 00000055 AE65:70AB
C:\USERS\SAROJ\TEST
├──test_dir_1_updated
└──test_dir_2
```

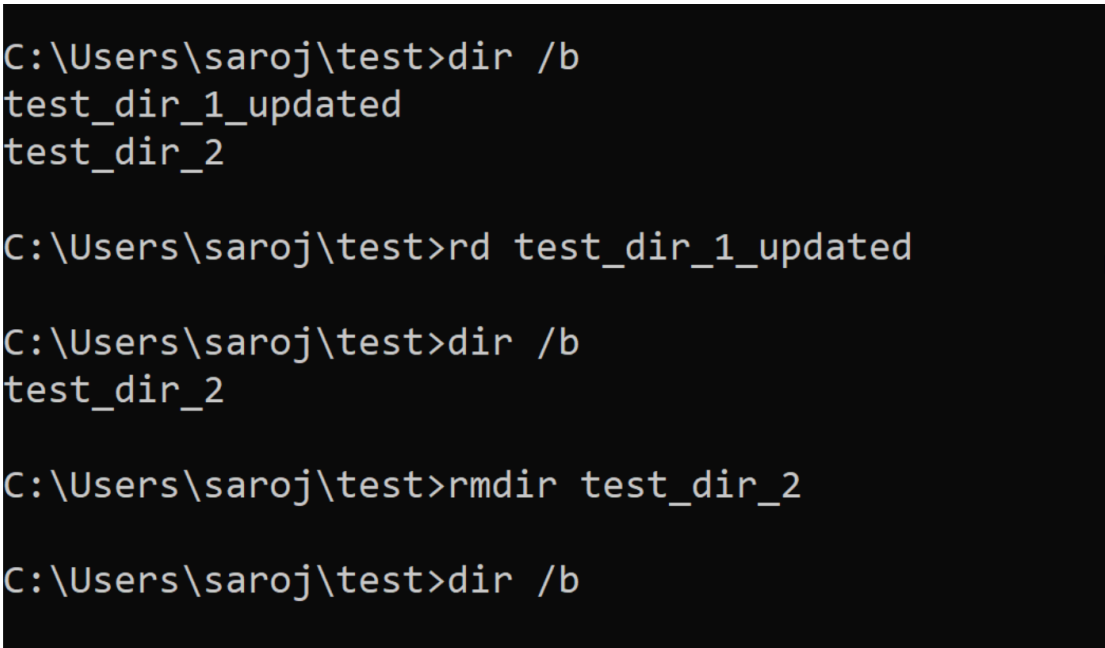
Figure 13: An example of using **tree** command.

4.6 rd or rmdir

rd or **rmdir** command is used to delete folder, if proper flags are specified it can also delete all the files and folder contained inside the specified folder.

syntax :

```
rd [<drive>:][<path>] [/s [/q]]
```



```
C:\Users\saroj\test>dir /b
test_dir_1_updated
test_dir_2

C:\Users\saroj\test>rd test_dir_1_updated

C:\Users\saroj\test>dir /b
test_dir_2

C:\Users\saroj\test>rmdir test_dir_2

C:\Users\saroj\test>dir /b
```

Figure 14: An example of using **rd** and **rmdir** command.

5. File Manipulation:

5.1 **edit**

edit command is used to start the MS-DOS Editor, which creates and changes ASCII text files.

syntax :

```
edit [/b] [/h] [/r] [/s] [/<nnn> [[<drive>:][<path>]<filename> [<filename2> [...]]
```

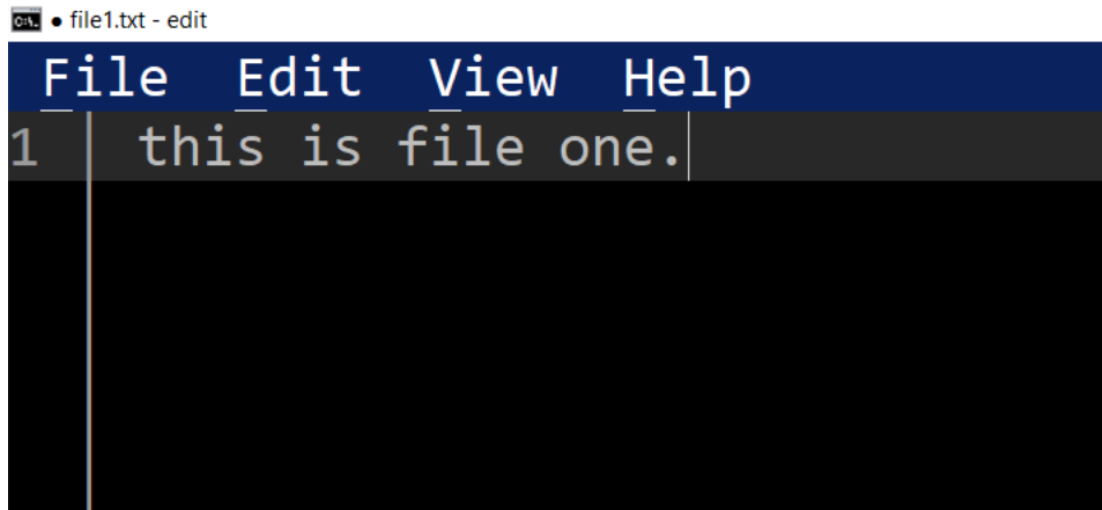


Figure 15: An example of using **edit** command.

5.2 **notepad**

notepad command or application can also be opened directly through cli interface of cmd.exe where we can create do different file operations.

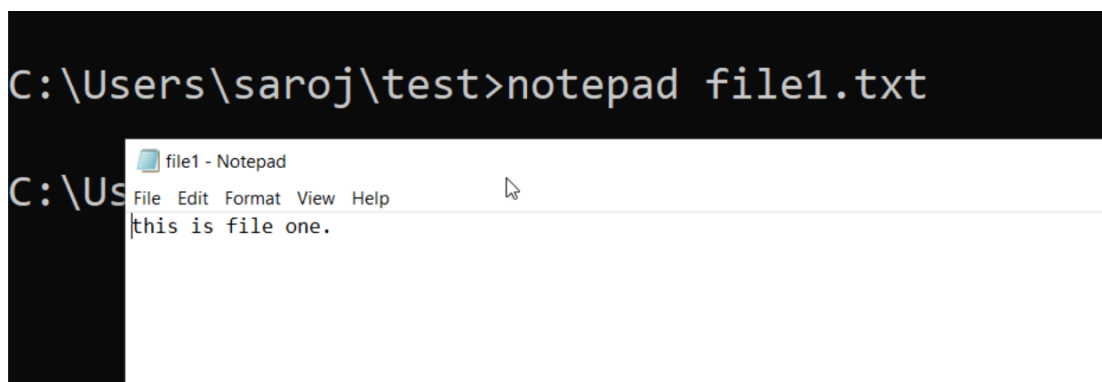


Figure 16: An example of using **opening notepad** using cli.

5.3 copy

copy command can be used to Copy one or more files from one location to another

syntax :

```
copy [/d] [/v] [/n] [/y | /-y] [/z] [/a | /b] <source> [/a | /b] [+<source> [/a | /b]
[+ ...]] [<destination> [/a | /b]]
```

```
C:\Users\saroj\test>dir /b
file1.txt

C:\Users\saroj\test>copy file1.txt file2.txt
1 file(s) copied.

C:\Users\saroj\test>dir /b
file1.txt
file2.txt
```

Figure 17: An example of using **copy** command.

5.4 del or erase

del or **erase** command can be used to delete one or more files from your filesystem.

syntax :

```
del [/p] [/f] [/s] [/q] [/a[:<attributes>] <names>
erase [/p] [/f] [/s] [/q] [/a[:<attributes>] <names>
```

```
C:\Users\saroj\test>dir /b
file1.txt
file2.txt

C:\Users\saroj\test>del file1.txt

C:\Users\saroj\test>dir /b
file2.txt

C:\Users\saroj\test>erase file2.txt

C:\Users\saroj\test>dir /b
```

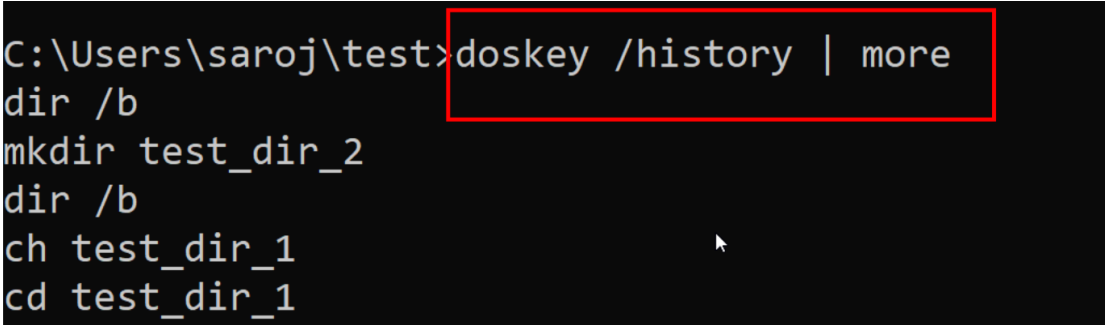
Figure 18: An example of using **del** and **erase** command.

5.5 doskey

doskey command calls Doskey.exe, which recalls previously entered command-line commands, edits command lines, and creates macros.

syntax :

```
doskey [/reinstall] [/listsize=<size>] [/macros:[all | <exename>] [/history] [/insert  
| /overstrike] [/exename=<exename>] [/macrofile=<filename>] [<macroname>=<text>]
```



A screenshot of a Windows command prompt window. The prompt is 'C:\Users\saroj\test>'. The command 'doskey /history | more' is entered and highlighted with a red rectangular box. Below the command, the following commands are listed: 'dir /b', 'mkdir test_dir_2', 'dir /b', 'ch test_dir_1', and 'cd test_dir_1'. A mouse cursor is visible over the 'ch test_dir_1' line.

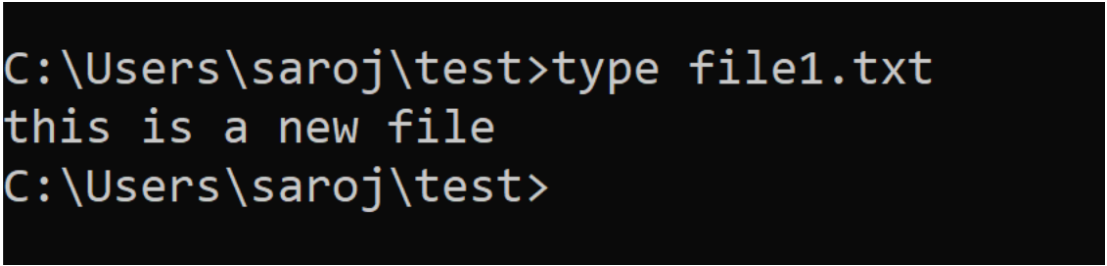
Figure 19: An example of using **doskey** command.

5.7 type

type command can be used to display the contents of a text file. We can use the type command to view a text file without modifying it.

syntax :

```
type [<drive>:][<path>]<filename>
```



A screenshot of a Windows command prompt window. The prompt is 'C:\Users\saroj\test>'. The command 'type file1.txt' is entered. The output of the command is displayed on the next line: 'this is a new file'. The prompt returns to 'C:\Users\saroj\test>'.

Figure 20: An example of using **type** command.

5.8 print

print command can be used to send a text file to a printer. A file can print in the background if you send it to a printer connected to a serial or parallel port on the local computer

syntax :

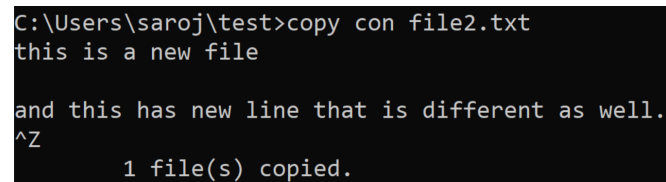
```
print [/d:<prntername>] [<drive>:][<path>]<filename>[ ...]
```

5.9 copy con

copy con command is used to create, write, or overwrite text and batch files directly from the Command Prompt

syntax :

```
copy con [<filename>]
```



```
C:\Users\saroj\test>copy con file2.txt
this is a new file

and this has new line that is different as well.
^Z
1 file(s) copied.
```

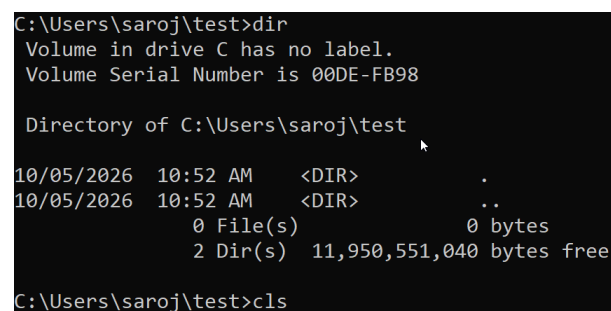
Figure 21: An example of using **copy con** command.

5.10 attrib

attrib command can be used to display, set, or remove attributes assigned to files or directories. If used without parameters, attrib displays attributes of all files in the current directory.

syntax :

```
attrib [{+|-}r] [{+|-}a] [{+|-}s] [{+|-}h] [{+|-}o] [{+|-}i] [{+|-}x] [{+|-}p] [{+|-}u] [{+|-}b] [<drive>:][<path>][<filename>] [/s [/d] [/l]]
```



```
C:\Users\saroj\test>dir
Volume in drive C has no label.
Volume Serial Number is 00DE-FB98

Directory of C:\Users\saroj\test

10/05/2026  10:52 AM    <DIR>          .
10/05/2026  10:52 AM    <DIR>          ..
               0 File(s)              0 bytes
               2 Dir(s)  11,950,551,040 bytes free

C:\Users\saroj\test>cls
```

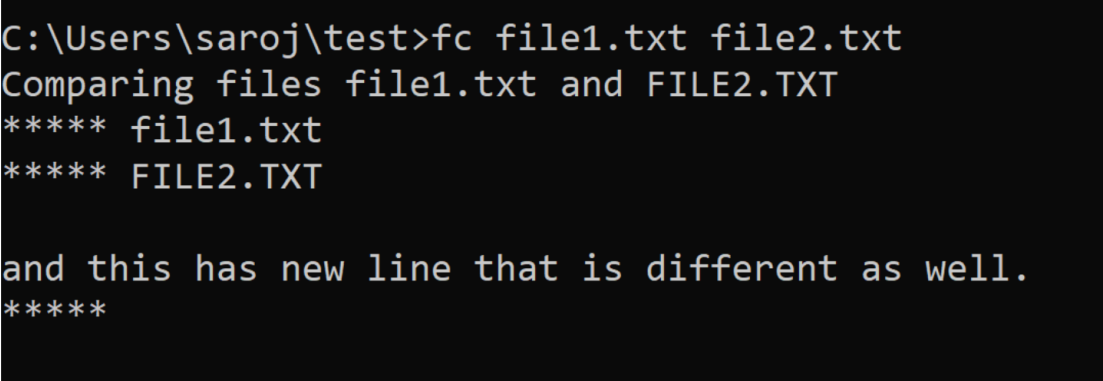
Figure 22: An example of using **attrib** command.

5.11 **fc**

fc command can be used to compare two files or sets of files and display the differences between them.

syntax :

```
fc /a [/c] [/l] [/lb<n>] [/n] [/off[line]] [/t] [/u] [/w] [/<nnnn>] [<drive1>:]  
[<path1>]<filename1> [<drive2>:][<path2>]<filename2>  
fc /b [<drive1:>][<path1>]<filename1> [<drive2:>][<path2>]<filename2>
```



```
C:\Users\saroj\test>fc file1.txt file2.txt  
Comparing files file1.txt and FILE2.TXT  
***** file1.txt  
***** FILE2.TXT  
  
and this has new line that is different as well.  
*****
```

Figure 23: An example of using **fc** command.

6. Use of Redirection, Filters, Pipes

6.1 Redirection input and output

6.1.1 output redirection (>)

The greater than operator is used to redirect the output of stdout into a file instead of displaying it into the screen.

example :

```
dir > file.txt
```

in the above example the all the files and folder's name inside the current directory will be added inside the file called file.txt.

6.1.2 output redirection append (>>)

The double greater than operator is used to redirect the output of stdout into a file instead of displaying it into the screen. But instead of replacing the items of the specified file it appends additional contents to it.

example :

```
"some more text" >> file.txt
```

in the above example the some more text will be appended to the file.txt where previously all the files and folder's name of the current directory were present.

6.1.2 input redirection (<)

The smaller than operator is used to input a file or stdin as input to a program.

example :

```
sort < names.txt
```

here in the example above, the contents of the file name.txt are passes as the input to the sort command using input redirection.

6.2 Use of Filters and Pipes

6.2.1 Pipe (|)

The pipe operator (|) is used to provide the output of one command as input to another command.

example :

```
dir | sort
```

here in the example above, the output of `dir` command is passed as the input to the `sort` command.

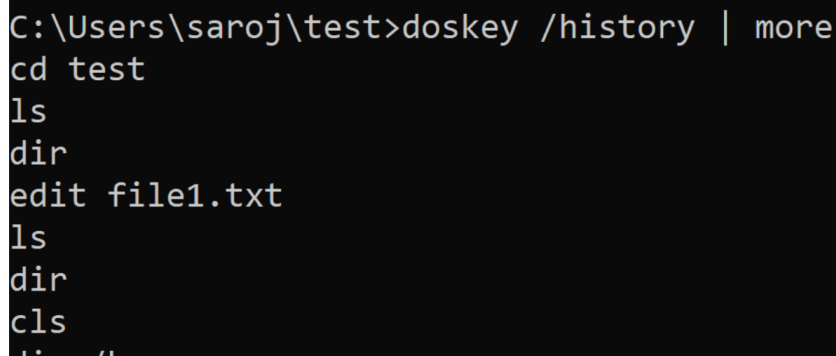
6.2.2 Filters commands

6.2.2.1 **more**

more command is used to limit the output of a program one screen at a time. It is specifically useful when the output of certain commands is more than a screen and is hard to fit in a single screen.

syntax :

```
<command> | more [/c] [/p] [/s] [/t<n>] [+<n>]  
more [[/c] [/p] [/s] [/t<n>] [+<n>]] < [<drive>:][<path>]<filename>  
more [/c] [/p] [/s] [/t<n>] [+<n>] [<files>]
```



```
C:\Users\saroj\test>doskey /history | more  
cd test  
ls  
dir  
edit file1.txt  
ls  
dir  
cls
```

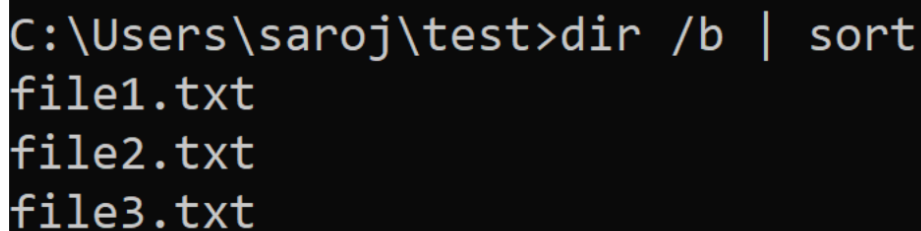
Figure 24: Using more filter command

6.2.2.2 **sort**

sort can be used to read input, sort data, and write the results to the screen or to a file using redirect operations like (`>`, `>>`).

syntax :

```
sort [/r] [/+<N>] [/m <kilobytes>] [/l <locale>] [/rec <characters>] [[<drive1>:]  
[<path1>]<filename1>] [/t [<drive2>:][<path2>]] [/o [<drive3>:][<path3>]<filename3>]
```



```
C:\Users\saroj\test>dir /b | sort  
file1.txt  
file2.txt  
file3.txt
```

Figure 25: Using sort command